
HungerHeal

How a request travels through each service

150 questions with answers

12 sections

Event-driven microservices · Go · Kafka · Redis

HungerHeal — how a request travels through each service

A file-by-file map of every service. For each one: what the entry points are, which file hands off to which, and what happens on the failure paths.

How to read the diagrams. Boxes are files. Arrows are "calls" or "passes the request to". Anything marked ▶ is a decision or a side effect worth remembering. Files are written as they appear on disk, so you can open them beside this.

Read section 1 first. All seven Node services share one skeleton — learn it once and each service is then only the two or three files that make it different.

Contents

Section	Service	Kind
1	The shared Node skeleton	—
2	One donation, end to end	all
3	api-gateway	edge
4	auth-service	HTTP + Mongo
5	donation-service	HTTP + Mongo + Kafka producer
6	geocoding-service	HTTP + Redis cache
7	agent-location-service	HTTP + Redis (Go)
8	assignment-engine	Kafka in/out (Go)
9	tracking-service	Kafka consumer + HTTP
10	notification-service	Kafka consumer + HTTP
11	analytics-service	Kafka consumer + Cassandra

1 — The shared Node skeleton

Every Node service has these same files doing the same jobs. Once you know this, each service below is just "the skeleton, plus X".

```
src/
├─ index.js           ▶ bootstrap: connect dependencies, THEN listen
├─ app.js             ▶ build the middleware chain (no port binding)
├─ config/
│   ├─ env.js         ▶ read + validate env vars, fail fast at startup
│   ├─ db.js          ▶ mongoose connection
│   └─ redis.js       ▶ redis client          (only where Redis is used)
├─ middleware/
│   ├─ requestContext.js ▶ x-trace-id in/out, attaches req.log
│   ├─ requireAuth.js   ▶ verify JWT → req.user; requireRole(...) gate
│   ├─ validate.js      ▶ schema check → 400 with field-level details
│   └─ errorHandler.js ▶ the ONE place every error is formatted
├─ routes/
│   ├─ *.routes.js     ▶ URL → middleware chain → controller
│   └─ *.schemas.js    ▶ the validation schemas
├─ controllers/
│   └─                 ▶ the actual work for one endpoint
├─ models/
│   └─                 ▶ mongoose schemas
└─ utils/
    ├─ jwt.js          ▶ signToken / verifyToken
    ├─ ApiError.js     ▶ typed errors (badRequest, notFound, forbidden...)
    └─ logger.js       ▶ structured JSON logging
```

The universal request pipeline

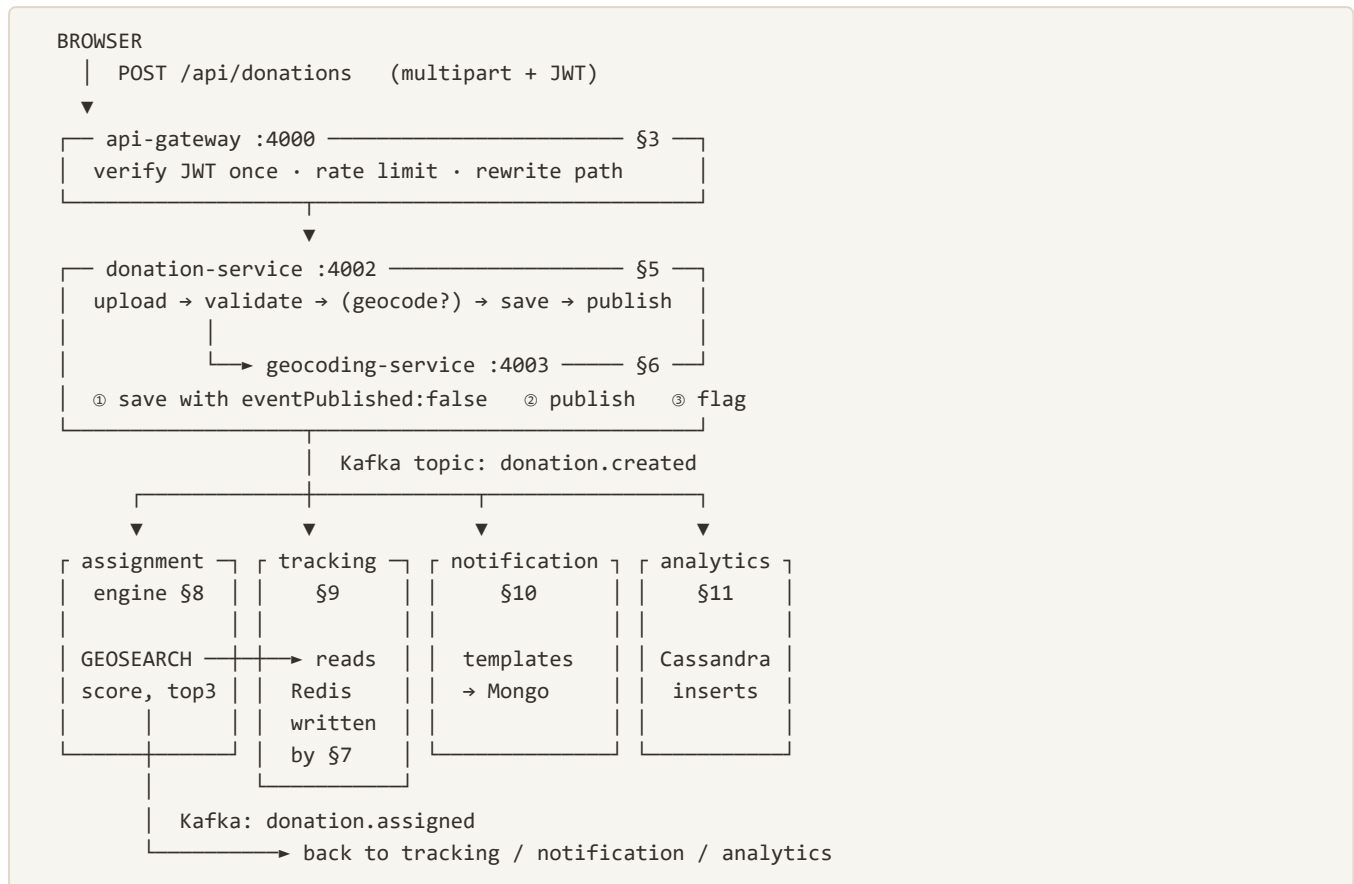


Why `index.js` and `app.js` are separate. `app.js` builds the Express app but never binds a port; `index.js` connects the database *first*, then listens. That split is what lets the tests drive the app in-process with supertest without binding a port or racing on one — and it means a service never accepts a request it cannot serve.

Why the error handler is last. Every failure, from any middleware or controller, funnels into one place. That is also where donation-service cleans up orphaned uploads — putting cleanup on the happy path missed every request that failed validation.

2 — One donation, end to end

Before the per-service detail, the whole journey. Each numbered step is a service section below.



Four consumer groups read `donation.created`. None of them knows the others exist, and donation-service knows about none of them.

3 — api-gateway (:4000)

Job: one front door. Verify the token once, rate limit, route to one of eight services.

```
browser
| POST /api/donations    Authorization: Bearer <jwt>
▼
index.js                ▶ starts the server
▼
app.js
|
|— helmet() · cors()
|
| (!) NO express.json() – deliberately.
|     Parsing the body would CONSUME the stream and the proxy
|     would forward an empty body downstream.
|
|— middleware/requestContext.js    ▶ x-trace-id
|
|— config/routes.js                ▶ match prefix, IN ORDER
|   /api/auth      → auth-service      basePath /auth
|   /api/donations → donation-service   basePath /donations
|   /uploads       → donation-service   basePath /uploads
|   /api/geo        → geocoding-service  basePath ''
|   /api/location   → agent-location     basePath ''
|   /api/engine     → assignment-engine  basePath ''
|   /api/monitoring → tracking-service   basePath /monitoring
|   /api/tracking   → tracking-service   basePath /tracking
|   /api/notify     → notification       basePath /notifications
|   /api/analytics  → analytics          (optional)
|
| (!) /api/monitoring MUST be registered before /api/tracking,
|     or /api/tracking swallows every monitoring request.
|
|— ratelimit                300/min general
|                           20/min on /api/auth, failures only
|
|— middleware/verifyToken.js
|   ▶ jwt.verify(token, secret, { algorithms: ['HS256'] })
|   ▶ sets req.user          ▶ auth:false routes skip this
|
|— createProxyMiddleware
|   ▶ pathRewrite: basePath + path      ← NOT a regex substitution
|   ▶ forwards x-trace-id, x-user-id, x-user-role
|   ▶ on error → 502 with an explanation
|   |
|   ▼
|   donation-service :4002 POST /donations
```

The bug worth remembering. `app.use(prefix, ...)` **strips** the prefix before the middleware runs, so `/api/auth/signup` was already `/signup` by the time the proxy saw it — and a `pathRewrite` regex matching `^/api/auth` matched nothing. 16 of 28 checks failed. The fix is to *prepend* the service's own `basePath` rather than substitute the public prefix.

GET /ready does not proxy: it fans out to all seven services and returns 503 if any is down.

4 — auth-service (:4001)

Job: users, passwords, JWTs. The only service that reads the users table.

```
index.js
├─ await connectDb()          config/db.js
├─ createApp().listen()       app.js
└─ SIGTERM → close server → disconnectDb()

— POST /auth/signup —————
▼
app.js → middleware/requestContext.js
▼
routes/auth.routes.js
|
├─ validateBody(signupSchema)
|   │ middleware/validate.js + routes/auth.schemas.js
|   └─ role discriminates the shape: an AGENT must send
|       │ capabilities { vehicleType, insulated, categories... }
|
▼
controllers/auth.controller.js  signup()
|
├─ bcrypt.hash(password, env.BCRYPT_ROUNDS)    10 rounds
├─ models/User.js      User.create(...)
|   └─ unique index on email settles concurrent signups
├─ utils/jwt.js        signToken(user)
|   └─ claims: sub, role, email, name, phone
└─ 201 { user, token }    └─ passwordHash never serialised

— POST /auth/login —————
controllers/auth.controller.js  login()
├─ User.findOne({ email })
├─ bcrypt.compare(password, user?.passwordHash ?? DUMMY_HASH)
|   └─ the dummy hash burns the same ~10 rounds when the email
|       │ does NOT exist, so timing cannot reveal who is registered
└─ 200 { user, token }    | 401 (identical either way)

— GET /auth/me —————
routes/auth.routes.js
├─ requireAuth          middleware/requireAuth.js
|   └─ utils/jwt.js verifyToken() → req.user
|
▼
controllers/auth.controller.js  me()
└─ User.findById(req.user.id)    └─ read FRESH, not from the token
```

Who else calls this service: only `agent-location-service`, and only on an agent's first contact, to mirror their capabilities into Redis. Nothing else ever asks "who is this?" — the signed token is the proof.

5 — donation-service (:4002)

Job: create and serve donations, and get `donation.created` onto Kafka without ever losing one. The most involved Node service.

```
index.js                                ▶ startup order matters
├─ await connectDb()                    config/db.js      (hard dependency)
├─ await connectProducer()              config/kafka.js  (BEST EFFORT –
│   ▶ a failure is logged, NOT fatal: donations are still
│   accepted and their events retried by the sweeper)
├─ startOutboxSweeper()                events/outbox.js  every 15s
└─ createApp().listen()

— POST /donations (multipart/form-data) —————
▼
routes/donation.routes.js
├─ requireAuth                        middleware/requireAuth.js
├─ requireRole('DONOR')
├─ uploadImages                      middleware/upload.js  (multer → disk)
│   (!) MUST run before validation: with multipart, req.body
│   does not exist until multer has parsed the stream
├─ handleUploadErrors
├─ validateBody(...)                routes/donation.schemas.js
▼
controllers/donation.controller.js  createDonation()
├─ ① no lat/lng sent?
│   └─ clients/geocodingClient.js → geocoding-service :4003
│       ▶ best effort: a failure leaves the donation
│       un-geocoded rather than rejecting it
├─ ② models/Donation.js  Donation.create({ ..., eventPublished: false })
│   ▶ DURABLE FIRST – this is the outbox
│   ▶ location uses `default: undefined` so an un-geocoded
│   donation has NO location field at all (a Point with
│   no coordinates is rejected by the 2dsphere index)
├─ ③ events/donationEvents.js  publishDonationCreated()
│   └─ config/kafka.js publish() → topic donation.created
│       key = donationId
├─ ④ if published → donation.eventPublished = true; save()
│   (!) a SKIPPED publish must count as failure, or the
│   sweeper never retries it
└─ 201 { donation, assignmentQueued, notice? }
    ▶ 201 even if Kafka was down
```

The outbox sweeper, and the reads

```
— background, every 15s —————  
events/outbox.js  sweepOutbox()  
  └─ Donation.find({ eventPublished: false })  
  └─ publishDonationCreated()    ▶ under the ORIGINAL traceId  
  └─ set the flag on success
```

```
— GET /donations —————  
controllers/donation.controller.js listDonations()  
  ▶ DONOR sees only their own; AGENT sees all  
  ▶ NOTE: donation.status here goes STALE after creation.  
    tracking-service is the authority; the UI reads that.
```

6 — geocoding-service (:4003)

Job: address ↔ coordinates, while spending as little of the paid API quota as possible.

```
— GET /geocode?address=... —————
  ▼
  routes/geocode.routes.js
  |— requireAuth
  |— validateQuery(geocodeQuerySchema)
  ▼
  controllers/geocode.controller.js  geocode()
  ▼
  cache/geocodeCache.js  cachedLookup(key, lookup)
  |
  |— cache/keys.js          ▶ normalise: lowercase, collapse whitespace
                           so "12 MG Road" and "12 mg road " are ONE key
  |
  |— readCache() → config/redis.js  GET
  |   |— HIT → return { ...value, cached: true }      ← no API call
  |   |— HIT of { notFound: true } → 404              ← negative cache
  |
  |— inFlight Map          ▶ SINGLE-FLIGHT
  |   ▶ 50 concurrent requests for the same uncached address
  |   make ONE provider call, not 50
  |
  |— MISS → providers/index.js
  |   |— providers/opencage.js  when OPENCAGE_API_KEY is set
  |   |— providers/offline.js  fallback, always works
  |   |
  |   |— writeCache()
  |       |— found → SETEX 30 days
  |       |— not found → SETEX 1 hour (shorter: a "no" is more
  |                               likely to become wrong than a "yes")
  |
  — GET /reverse?lat=&lng= —————
    same path, keyed on rounded coordinates

  — GET /stats —————
    cache hit rate + provider quota counter
```

Why Redis here is a cache, not a system of record. A miss costs an API call, not correctness — which is why `/ready` stays **200** when Redis is down here, and **503** in agent-location-service where Redis is the data.

7 — agent-location-service (:4004, Go)

Job: where every on-shift agent is, right now. The highest-frequency write in the system.

```
cmd/server/main.go
├ config.Load()                internal/config/config.go
├ store.New(redisURL, agentTTL) internal/store/redis.go
├ api.Routes(handlers, secret) internal/api/router.go
├ go runReaper(...)            ▶ background goroutine
└ server.ListenAndServe()

— POST /agents/location {lat,lng} —————
▼
internal/api/router.go
└ agentOnly( ... )            internal/httpx/middleware.go
  ▶ trace id → context
  ▶ verify JWT, pin HS256
  ▶ require role AGENT
▼
internal/api/handlers.go UpdateLocation()
├ decode body; validateCoordinates()
│   ▶ lat/lng are *float64 POINTERS, so a missing field is
│   told apart from a real 0 (the equator / Greenwich)
├ ensureCapabilities()         ← the Phase 13 fix
│   └ store.HasCapabilities()
│       HEXISTS agent:<id>:caps updatedAt
│       ▶ NOT plain EXISTS – a hash created by the
│       availability toggle is a stub, not a mirror
│   └ internal/authclient/client.go → auth-service /auth/me
├ store.SaveCapabilities()     full mirror, writes updatedAt
└ store.SavePartialProfile()   auth down: name/phone only,
                                NO updatedAt → stays retryable

├ store.UpsertLocation()      one Redis pipeline:
│   GEOADD agents:live <lng> <lat> <agentId>
│   SET    agent:<id>:alive 1 EX 120
│   (!) GEOADD takes LONGITUDE FIRST. Reversing them is silent.
└ internal/httpx/respond.go   JSON 200 { expiresInSeconds: 120 }
```

The other endpoints, and the reaper

```
— the other endpoints —————
POST /agents/availability → ensureCapabilities() FIRST, then
                           HSET caps available
POST /agents/offline      → ZREM agents:live + DEL alive
GET  /agents/nearby       → GEOSEARCH (what the engine would see)

— background goroutine —————
runReaper → store.Reap()
  ▶ a sorted-set MEMBER cannot expire, only a whole key.
    So: the alive key expires (checked on every read), and the
    reaper removes orphaned geo members so the set cannot grow
    without bound.
```

The four Redis keys

agents:live	GEO set	every position, one member per agent
agent:<id>:caps	HASH	capabilities, rating, contact, updatedAt
agent:<id>:load	counter	pending pickups (INCR/DECR, atomic)
agent:<id>:alive	string+TTL	heartbeat, 120s, refreshed every 20s

`assignment-engine` **reads these keys directly** — the one deliberate boundary exception in the system, because an HTTP hop on the hot path would cost more than it is worth. The key layout is a contract between the two services.

8 — assignment-engine (:4005, Go)

Job: the product's whole thesis. Kafka in, Kafka out; it owns no database.

```
cmd/engine/main.go
├─ candidates.NewFinder(redisURL)    internal/candidates/redis.go
├─ offers.New(client, dedupTTL)       internal/offers/store.go
├─ events.NewProducer(brokers)        internal/events/kafka.go
├─ engine.Engine{...}                 internal/engine/engine.go
├─ go watcher.Run(ctx)                internal/engine/watcher.go
├─ go consume(donation.created)
└─ go consume(donation.timeout)

— Kafka: donation.created —————
▼
internal/events/kafka.go    Consumer fetches the message
▼
cmd/engine/main.go
├─ parse → events.DonationCreated    internal/events/types.go
├─ eventId read from the BODY, not the header
│   (!) a REPLAYED message carries no headers – reading the
│       header there caused a donation to be assigned twice
└─ dedup: SETNX processed:<eventId>    internal/offers/store.go
    └─ already seen → commit and move on
    └─ on failure → RELEASE the marker, or the retry skips it
▼
internal/engine/engine.go    Handle()
├─ no coordinates? → publish donation.unassigned NOT_GEOCODED
│   └─ announced, never silently dropped
└─
▼
offerRound()                — continued below —
```

The scoring round — `internal/engine/engine.go` `offerRound()`

The heart of the system. Everything above exists to get here safely.

```

offerRound()
├─ internal/scoring/urgency.go
│   RadiusLadderKm(category)  HIGH 5→12→25→50 km
│                               MED 5→10→20→35
│                               LOW 5→8→12→20
│   ResponseTimeout(category) 90s / 3min / 5min
├─ for each radius on the ladder:
│   └─ internal/candidates/redis.go Nearby()
│       GEOSEARCH agents:live BYRADIUS <r> km ASC
│       ▶ skips anyone whose agent:<id>:alive has expired
│       ▶ reads caps + load for each hit
│   └─ internal/scoring/score.go RankAgents()
│       └─ HARD FILTERS, before any scoring:
│           ▶ excluded (already declined/ignored)
│           ▶ !Available
│           ▶ !Capabilities.Handles(category)
│       └─ ScoreAgent() for everyone left:
│           DistanceScore 1 - dist/radius      × 0.35
│           CategoryScore compatibility.go     × 0.25
│           LoadScore     1/(1+pending)        × 0.20
│           RatingScore    rating/5             × 0.15
│                               total out of 0.95
│       └─ sort: score ▶ then distance ▶ then agentId
│           (the id tie-break makes it reproducible)
│   └─ nobody eligible? → next radius up the ladder
│   └─ TopN(3)
│   └─ internal/offers/store.go
│       donation:<id>:offer      the ranks + full breakdowns
│       ZADD offers:deadlines <expiresAt> <donationId>
│   └─ internal/events/kafka.go Publish donation.assigned
│       ▶ all three offered IN PARALLEL, one event
└─ ladder exhausted —————
    publish donation.unassigned with a reason:
        NO_AGENTS_FOUND • NO_ELIGIBLE_AGENTS • ALL_AGENTS_EXHAUSTED

```

The HTTP side and the timeout loop

```
— HTTP side (what the agent's UI calls) —————  
internal/api/handlers.go  
GET  /offers/:id          the offer as the agent sees it  
POST /offers/:id/accept    SETNX donation:<id>:claimed  
                                ▶ winner 200 · losers 409  
POST /offers/:id/reject    adds them to donation:<id>:declined
```

```
— background goroutine —————  
internal/engine/watcher.go  Run()  
| ZRANGEBYSCORE offers:deadlines 0 <now>  
| publish donation.timeout  
└ engine.HandleTimeout()  
  | Offers.Declined() → the exclusion set  
  | (!) without it, a re-score finds the IDENTICAL top 3 and  
  | offers it back to the people who just ignored it  
  └ offerRound() again, at the next radius
```

9 — tracking-service (:4006)

Job: the authority on donation status. Two completely separate entry points — this is the clearest example of a service that both consumes and serves.

```
== ENTRY A: Kafka (all six topics) ==  
  
index.js → events/consumer.js  startConsumer(topics, handlers)  
  | groupId: tracking-service    ▶ its OWN group → full stream  
  | fromBeginning: true  
  |  
  ▼  
events/consumer.js  handleMessage()  
  | JSON.parse  
  |   ▶ unparseable → log and COMMIT (a poison message would  
  |   otherwise block the whole partition forever)  
  |  
  | eventId + traceId from the BODY (header only as fallback)  
  |  
  | beginProcessing(eventId)    SETNX    config/redis.js  
  |   ▶ already processed → return, commit, done  
  |  
  | events/handlers.js  buildHandlers()[topic](event)  
  |   |  
  |   | domain/statusMachine.js  applyEvent(current, event)  
  |   |   ▶ PURE function, imports nothing  
  |   |   ▶ checks "already in this status" BEFORE legality,  
  |   |   so a routine redelivery is a silent no-op  
  |   |   ▶ terminal states accept nothing further  
  |   |  
  |   | models/DonationStatus.js  
  |   |   upsert + push onto the embedded timeline[]  
  |   |   + snapshot assignedAgentName / Phone  
  |   |   (no joins across services – so it is copied)  
  |   |  
  |   └ on error → abandonProcessing(eventId) ▶ RELEASE the marker  
  |       then rethrow ▶ offset NOT committed → redelivered
```

Entry B — HTTP

```
routes/tracking.routes.js      requireAuth
├ GET /stats/summary           ┌
├ GET /                       └─> controllers/tracking.controller.js
├ GET /:donationId             ┌
└ POST /:donationId/collected  └─> models/DonationStatus.js
    requireRole('AGENT')
    └─> events/producer.js  publish donation.collected
        └─> 202, not 200: the status changes only once
            the service consumes its OWN event back

routes/monitoring.routes.js      requireAuth + requireRole('ADMIN')
├ GET /stats
├ GET /donations
└ GET /donations/:donationId    └─> full score breakdown per candidate
    └─> EVERY route here is a GET. There is no write endpoint,
        deliberately – no assign button, no override.
```

10 — notification-service (:4007)

Job: turn events into messages a human would want to read.

```
== Kafka ==
events/consumer.js  (same skeleton as §9: dedup, release, rethrow)
  subscribes: donation.assigned · accepted · unassigned
    ▼
events/handlers.js
  | domain/templates.js
  |   ▶ PURE function: event → { title, body, priority }
  |   ▶ priority HIGH on the ones that would justify waking a
  |     phone – carried, but nothing delivers push yet
  |
  | models/Notification.js  insert
  |   ▶ unique PARTIAL index on (sourceEventId, recipientId)
  |     makes a duplicate physically IMPOSSIBLE even if the
  |     Redis dedup let one through – a second line of defence
  |   ▶ partial, because not every notification comes from an
  |     event; without the filter every null would collide
  |
== HTTP ==
routes/notification.routes.js      requireAuth
  | GET  /unread-count
  | GET  /                               → controllers/notification.controller.js
  | POST /read-all                     ↳ models/Notification.js
  | PATCH /:id/read
  |   ▶ scoped to req.user.id – you can only read your own
```

11 — analytics-service (:4008, optional)

Job: the historical record. Runs only under the `analytics` Docker profile.

```
== Kafka ==
events/consumer.js
  | subscribes to ALL SIX topics
  | fromBeginning: true
  |   ▶ THIS is the payoff of an event log: written LAST, it
  |     replayed 133 events across 4 days – including days
  |     before the service existed. No migration was written.
  |   ▼
events/handlers.js → config/cassandra.js
  |
  | | INSERT donation_events      partition: donationId
  | |                             cluster:   (occurredAt, eventId)
  | |   ▶ eventId is a TIE-BREAK: two events in the same
  | |     millisecond would otherwise overwrite each other
  | |
  | | | INSERT events_by_day      partition: day
  | | |   ▶ THE SAME EVENTS, STORED TWICE, on purpose.
  | | |     donation_events cannot answer "what happened today?"
  | | |     without scanning every partition. In Cassandra you
  | | |     duplicate data to buy query performance.
  | | |
  | | | INSERT assignment_outcomes  one row per decision
  | | |
  | | | UPDATE agent_totals / daily_totals  COUNTER columns
  | | |   (!) counters are NOT idempotent – a replay double-counts.
  | | |     That is exactly why dedup runs before this, and why
  | | |     counters live in their own tables (Cassandra forbids
  | | |     counter and non-counter columns in one table).
  | |
  | ▶ Writes go individually, never in a multi-partition BATCH:
  |   that is not an optimisation and not a transaction – it just
  |   makes one coordinator responsible for writes to many nodes.

== HTTP ==
routes/analytics.routes.js → controllers/analytics.controller.js
  ▶ read-only aggregate queries
```

Revision checklist

Cover the page and answer these. If you can, you know the file map.

1. Which file makes the trace id, and which file formats every error?
2. Why are `index.js` and `app.js` separate?
3. In donation-service, why must multer run **before** validation?
4. Name the three steps of the outbox, in order.
5. Which file decides that a closer agent can lose? (`scoring/score.go` — hard filters, then the four weighted terms)
6. Why does the engine read the `eventId` from the body and not the header?
7. What does the reaper goroutine exist to work around?
8. In tracking-service, what happens to the dedup marker when a handler throws?
9. Why does `POST /tracking/:id/collected` return 202 rather than 200?
10. Why does `events_by_day` hold the same rows as `donation_events` ?